

COMP0264 Generative AI

Exam Revision Notes: Variational Autoencoders (VAEs)

Auto-generated from Lecture Slides & Recordings

UCL 2025–26

Summary: This document covers all examinable content on Variational Autoencoders (VAEs) for COMP0264, including: review of standard autoencoders, sparse and denoising autoencoders, the VAE probabilistic framework, ELBO derivation, the reparameterisation trick, and the training/generation pipeline. Content is drawn from lecture slides (Slides 1–43) and two lecture recordings (16 & 17 March 2026). **Note:** The lecturer explicitly stated that Quantisation is NOT on the exam.

Contents

1 Overview	2
2 Standard Autoencoders (Background)	2
2.1 Sparse Autoencoders	3
2.2 Denoising Autoencoders	3
3 Variational Autoencoders: Core Idea	3
4 VAE Architecture	4
5 The Likelihood and Its Intractability	5
6 Deriving the Evidence Lower Bound (ELBO)	5
6.1 Full Derivation of the ELBO	6
6.2 Why It Is a Lower Bound	6
6.3 True Posterior vs. Approximate Posterior	6
7 Maximising the ELBO	7
7.1 Per-Data-Point ELBO	7
7.2 Choice of Prior: Standard Normal	7
7.3 Evaluating the Two Terms of the ELBO	8
8 The Reparameterisation Trick	8
8.1 The Problem	9
8.2 The Solution	9
8.3 Full Mini-Batch Objective	9
9 Training Procedure Summary	10
10 Generation After Training	10
11 Conditional VAE and Extensions	11

12 Potential Exam Questions	11
13 References	13
13.1 Cited on Slides	13
13.2 Mentioned in Lectures Only	13

1 Overview

This topic covers **Variational Autoencoders (VAEs)**, one of the most popular deep generative models. The material was taught across two lectures (16 March and 17 March 2026) and is based on the slide set **GenAI2526_VAEs.pdf** (43 slides).

The logical flow of the topic is:

1. Review of standard autoencoders and their limitations.
2. Motivation for learning a *probability distribution* over the latent space (not just a compressed vector).
3. The VAE architecture: encoder $q(z|x, \phi)$ and decoder $p(x|z, w)$.
4. Derivation of the Evidence Lower Bound (ELBO).
5. The reparameterisation trick for end-to-end training.
6. Training procedure and generation after training.

EXAM ALERT:

Quantisation NOT on the exam The lecturer explicitly and repeatedly stated: “Quantisation will not be in the exam.” Everything else covered in the module **is** examinable.

2 Standard Autoencoders (Background)

Definition: Autoencoder

An autoencoder is a neural network with the **same number of outputs as inputs**, trained to produce an output $\mathbf{y}$ that is close to the input $\mathbf{x}$. An internal layer provides a hidden (latent) representation $\mathbf{z}(\mathbf{x})$ for each input.

The network has two parts:

- **Encoder** F_1 : maps $\mathbf{x} \mapsto \mathbf{z}(\mathbf{x})$.
- **Decoder** F_2 : maps $\mathbf{z} \mapsto \mathbf{y}(\mathbf{z})$.

The goal of training is to minimise $\|\mathbf{y} - \mathbf{x}\|^2$.

Why constraints are needed

Without constraints, the network simply learns the identity mapping (copying inputs to outputs). Common constraints include:

- **Bottleneck**: require $\dim(\mathbf{z}) = M \ll D = \dim(\mathbf{x})$.
- **Sparsity**: penalise activations in the latent layer.
- **KL divergence**: push activations towards low activation probability.

Why autoencoders learn good representations

The “pressure” of the bottleneck encourages:

- Removing noise and redundancy.
- Capturing dominant structures (patterns).

- Learning compact features — essentially performing *information compression*.

LECTURE INSIGHT:

Compression analogy The lecturer emphasised: an autoencoder is “literally a compression of information.” The bottleneck forces the network to capture only the most important patterns, discarding noise and redundancy.

2.1 Sparse Autoencoders

L_1 Regularised Autoencoder

Instead of limiting the number of hidden nodes, we use a regulariser to encourage sparsity. With an L_1 regulariser:

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda \sum_{k=1}^K |z_k|$$

where $E(\mathbf{w})$ is the unregularised reconstruction error.

2.2 Denoising Autoencoders

Denoising Autoencoder

Given an input vector $\mathbf{x}_n$, we corrupt it with noise to produce $\tilde{\mathbf{x}}_n$. The network is trained to reconstruct the **original noise-free** input by minimising:

$$E(\mathbf{w}) = \sum_{n=1}^N \|\mathbf{y}(\tilde{\mathbf{x}}_n, \mathbf{w}) - \mathbf{x}_n\|^2$$

This forces the network to learn robust features rather than memorising inputs.

LECTURE INSIGHT:

Denoising autoencoder paper The foundational paper on denoising autoencoders was published in JMLR 2010, from the Toronto group (Vincent, Larochelle, Bengio, et al.). The lecturer noted this is “the usual suspects” from Toronto.

STUDENT Q&A: If you don’t enforce the bottleneck, how does the model throw away information?

You *must* enforce some form of constraint — either a bottleneck, sparsity regularisation, denoising, or a variational constraint (which is exactly what VAEs provide).

3 Variational Autoencoders: Core Idea

VAE: Key Difference from Standard AE

A Variational Autoencoder (VAE) is essentially a simple autoencoder, except that we now **define a probability distribution over the latent space**, rather than mapping to a deterministic latent vector.

A VAE comprises two neural networks with *independent parameters* trained *jointly*:

- **Encoder network:** maps data vector $\mathbf{x}$ to a *distribution* in latent space.
- **Decoder network:** maps latent vector $\mathbf{z}$ back to data space.

LECTURE INSIGHT:

Why learn a distribution instead of a vector? The lecturer stressed: “The big difference with respect to standard autoencoders is that $\mathbf{z}$ is *sampled* from a distribution $q(\mathbf{z}|\mathbf{x}, \phi)$.” If we learn a distribution over the latent space, we can then *generate new data* by sampling from that distribution. A standard autoencoder is merely a feature compressor; a VAE is a **true generative model**.

Why “Variational”?

The word “variational” comes from **variational inference**: a technique where we approximate an unknown probability distribution by optimising a simpler, parameterised one. “Variational” means the model is learning by solving a *distribution approximation problem*, not just a compression problem. Because learning a distribution forces the latent space to be “organised” and “well-behaved,” VAEs are true generative models.

LECTURE INSIGHT:

Origins in physics The lecturer noted that variational inference is “used quite a lot in physics and applied maths.” The core idea is to approximate an unknown probability distribution using a simpler parametric family.

4 VAE Architecture

Architecture

The VAE architecture consists of:

Encoder (parameterised by ϕ): takes input $\mathbf{x}$ and outputs the parameters $\mu_j(\mathbf{x}, \phi)$ and $\sigma_j^2(\mathbf{x}, \phi)$ of a Gaussian distribution over $\mathbf{z}$.

Latent variable $\mathbf{z}$: sampled from $q(\mathbf{z}|\mathbf{x}, \phi)$.

Decoder (parameterised by $\mathbf{w}$): takes $\mathbf{z}$ and outputs $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$.

Encoder Distribution

The typical choice for the encoder is a Gaussian distribution with a **diagonal covariance matrix**:

$$q(\mathbf{z}|\mathbf{x}, \phi) = \prod_{j=1}^M \mathcal{N}(z_j \mid \mu_j(\mathbf{x}, \phi), \sigma_j^2(\mathbf{x}, \phi))$$

where j indexes the dimensions of $\mathbf{z} \in \mathbb{R}^M$, and μ_j, σ_j^2 are the outputs of the encoder neural network taking $\mathbf{x}$ as input.

LECTURE INSIGHT:

μ and σ are not sampled The lecturer clarified an important subtlety: “ μ and σ are not sampled — they are calculated directly using the encoder network for each element of the training set (given the current parameters ϕ). The only sampled values are ϵ .” Every forward pass through the encoder gives you μ_j and σ_j^2 ; then $\mathbf{z}$ is obtained by sampling.

LECTURE INSIGHT:

Each input gives different μ, σ . A student asked whether μ and σ are different for every forward pass. The lecturer confirmed: “Yes. Each data point $\mathbf{x}_n$ produces its own $\mu_j(\mathbf{x}_n)$ and $\sigma_j(\mathbf{x}_n)$.”

5 The Likelihood and Its Intractability

Likelihood Function

The likelihood function for the latent-variable model is:

$$p(\mathbf{x}|\mathbf{w}) = \int p(\mathbf{x}|\mathbf{z}, \mathbf{w}) p(\mathbf{z}) d\mathbf{z}$$

where $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$ is defined by the decoder neural network. This integral is **intractable** because $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$ is a complex non-linear function.

Why Maximise Likelihood?

We want to maximise $p(\mathbf{x}|\mathbf{w})$ because:

- It means the model learns the true data distribution.
- We force the model to assign high probability to samples that look like real training data.

Likelihood measures how well the model’s parameters $\mathbf{w}$ explain the observed data.

LECTURE INSIGHT:

Why neural networks? The lecturer asked the class why we use neural networks for $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$. The answer: because we need a **universal function approximator**. The universal approximation theorem guarantees that neural networks can approximate any continuous function, which is why they are used to parameterise the intractable likelihood.

6 Deriving the Evidence Lower Bound (ELBO)

This is the mathematical core of VAEs and was emphasised heavily in both lectures.

ELBO Decomposition (Key Result)

For any arbitrary distribution $q(\mathbf{z}|\phi)$, the following holds:

$$\ln p(\mathbf{x}|\mathbf{w}) = \mathcal{L}(\mathbf{w}, \phi) + \text{KL}(q(\mathbf{z}|\phi) \parallel p(\mathbf{z}|\mathbf{x}, \mathbf{w}))$$

where the **Evidence Lower Bound (ELBO)** $\mathcal{L}$ is:

$$\mathcal{L}(\mathbf{w}, \phi) = \int q(\mathbf{z}|\phi) \ln \left\{ \frac{p(\mathbf{x}|\mathbf{z}, \mathbf{w}) p(\mathbf{z})}{q(\mathbf{z}|\phi)} \right\} d\mathbf{z}$$

and the KL divergence is:

$$\text{KL}(q(\mathbf{z}|\phi) \parallel p(\mathbf{z}|\mathbf{x}, \mathbf{w})) = \int q(\mathbf{z}|\phi) \ln \left\{ \frac{q(\mathbf{z}|\phi)}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} \right\} d\mathbf{z}$$

6.1 Full Derivation of the ELBO

Step-by-step derivation

Start from the log-likelihood and take the expectation under $q(\mathbf{z}|\mathbf{x}, \phi)$:

$$\ln p(\mathbf{x}|\mathbf{w}) = \mathbb{E}_{q(\mathbf{z}|\mathbf{x}, \phi)}[\ln p(\mathbf{x}|\mathbf{w})] \quad (1)$$

$$= \mathbb{E}_q \left[\ln \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} \right] \quad (\text{since } p(\mathbf{x}|\mathbf{w}) = \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})}) \quad (2)$$

$$= \mathbb{E}_q \left[\ln \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{q(\mathbf{z}|\mathbf{x}, \phi)} \cdot \frac{q(\mathbf{z}|\mathbf{x}, \phi)}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} \right] \quad (\text{multiply and divide by } q) \quad (3)$$

$$= \mathbb{E}_q \left[\ln \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{q(\mathbf{z}|\mathbf{x}, \phi)} \right] + \mathbb{E}_q \left[\ln \frac{q(\mathbf{z}|\mathbf{x}, \phi)}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} \right] \quad (\ln(ab) = \ln a + \ln b) \quad (4)$$

$$= \underbrace{\int q(\mathbf{z}|\mathbf{x}, \phi) \ln \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{q(\mathbf{z}|\mathbf{x}, \phi)} d\mathbf{z}}_{\mathcal{L}(\mathbf{x}|\mathbf{w}, \phi) \text{ (ELBO)}} + \underbrace{\int q(\mathbf{z}|\mathbf{x}, \phi) \ln \frac{q(\mathbf{z}|\mathbf{x}, \phi)}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} d\mathbf{z}}_{\text{KL}(q||p) \geq 0} \quad (5)$$

LECTURE INSIGHT:

The derivation is “very logical” The lecturer walked through each step slowly and emphasised: “People like to make things complicated, but this is just: (1) write $p(\mathbf{x}|\mathbf{w})$ using Bayes’ rule, (2) multiply and divide by q , (3) split the logarithm of a product into a sum. That is all.” He also noted that the approach in the slides follows the same presentation as the Kingma & Welling monograph (2019), which he considers the easiest way to understand VAEs.

6.2 Why It Is a Lower Bound

ELBO is a lower bound on the log-likelihood

Since the KL divergence is **non-negative** (and equals zero if and only if $q(\mathbf{z}|\mathbf{x}, \phi) = p(\mathbf{z}|\mathbf{x}, \mathbf{w})$):

$$\mathcal{L}(\mathbf{x}|\mathbf{w}, \phi) = \ln p(\mathbf{x}|\mathbf{w}) - \text{KL}(q(\mathbf{z}|\mathbf{x}, \phi) || p(\mathbf{z}|\mathbf{x}, \mathbf{w})) \leq \ln p(\mathbf{x}|\mathbf{w})$$

This is why $\mathcal{L}$ is called the **Evidence Lower Bound**: the log-evidence $\ln p(\mathbf{x}|\mathbf{w})$ can never be less than $\mathcal{L}$.

LECTURE INSIGHT:

Why it is called “lower bound” The lecturer explained: “It becomes a lower bound because the KL divergence is always non-negative. The lowest the log-likelihood can go is when $\text{KL} = 0$, at which point $\mathcal{L} = \ln p(\mathbf{x}|\mathbf{w})$.” The gap between the ELBO and the true log-likelihood is called the **tightness of the bound**.

6.3 True Posterior vs. Approximate Posterior

Distinction between true and approximate posterior

- The **true posterior** $p(\mathbf{z}|\mathbf{x}, \mathbf{w})$ is defined by Bayes' rule. It is “true” in the sense of being mathematically exact for the model.
- The **approximate posterior** $q(\mathbf{z}|\mathbf{x}, \phi)$ is restricted to a simpler, tractable form (a diagonal Gaussian — non-zero entries only on the main diagonal of the covariance matrix). It is learned by optimisation.

7 Maximising the ELBO

Dual Optimisation Property (Key Insight)

Maximising the ELBO $\mathcal{L}(\mathbf{x}|\mathbf{w}, \phi)$ with respect to both $\mathbf{w}$ and ϕ **simultaneously achieves two goals**:

1. It (approximately) maximises the marginal likelihood $p(\mathbf{x}|\mathbf{w}) \Rightarrow$ **better generative model**.
2. It minimises $\text{KL}(q(\mathbf{z}|\mathbf{x}, \phi) \| p(\mathbf{z}|\mathbf{x}, \mathbf{w})) \Rightarrow$ **better approximate posterior** q .

LECTURE INSIGHT:

“Two optimisations for the price of one” The lecturer was visibly enthusiastic about this result: “This is kind of the genius of the VAE — by maximising one objective (the ELBO), you simultaneously get a better p (generative model) and a better q (encoder). It’s quite powerful.” He attributed this insight to Kingma and Welling’s ability to find a formulation with this elegant dual property.

7.1 Per-Data-Point ELBO

ELBO for a single data point $\mathbf{x}_n$

For a training set $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, the contribution of data point $\mathbf{x}_n$ to the lower bound is:

$$\mathcal{L}_n(\mathbf{w}, \phi) = \int q(\mathbf{z}_n|\mathbf{x}_n, \phi) \ln p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w}) d\mathbf{z}_n - \text{KL}(q(\mathbf{z}_n|\mathbf{x}_n, \phi) \| p(\mathbf{z}_n))$$

7.2 Choice of Prior: Standard Normal

Standard VAE uses $\mathcal{N}(\mathbf{0}, \mathbf{I})$ as prior

In the standard VAE, we use the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (zero mean, unit diagonal covariance, uncorrelated) instead of $p(\mathbf{z}|\mathbf{x}, \phi)$. This gives:

$$\mathcal{L}_n(\mathbf{w}, \phi) = \int q(\mathbf{z}_n|\mathbf{x}_n, \phi) \ln p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w}) d\mathbf{z}_n - \text{KL}(q(\mathbf{z}_n|\mathbf{x}_n, \phi) \| p(\mathbf{z}_n))$$

Using $p(\mathbf{z}|\mathbf{x}, \phi)$ instead would give a **Conditional VAE**, which requires $\mathbf{x}$ values at generation time.

LECTURE INSIGHT:

Why the standard normal prior? The lecturer gave three reasons for using $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$:

1. It defines a **single global latent space** for all inputs (no conditioning on $\mathbf{x}$).
2. At generation time, we do **not need the encoder** — we simply sample from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and feed into the decoder.
3. It produces **smooth representations** and helps avoid overfitting.

With a conditional prior $p(\mathbf{z}|\mathbf{x})$, you would need input $\mathbf{x}$ values to generate, defeating the purpose of a generative model.

7.3 Evaluating the Two Terms of the ELBO**Monte Carlo estimation of reconstruction term**

The first term (reconstruction) is approximated using a Monte Carlo estimator:

$$\int q(\mathbf{z}_n|\mathbf{x}_n, \phi) \ln p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w}) d\mathbf{z}_n \approx \frac{1}{L} \sum_{l=1}^L \ln p(\mathbf{x}_n | \mathbf{z}_n^{(l)}, \mathbf{w})$$

where $\{\mathbf{z}_n^{(l)}\}$ are L samples drawn from $q(\mathbf{z}_n|\mathbf{x}_n, \phi)$.

Analytical KL divergence

The second term (KL divergence from q to the standard normal prior) can be computed **analytically**:

$$\text{KL}(q(\mathbf{z}|\mathbf{x}, \phi) \parallel p(\mathbf{z})) = -\frac{1}{2} \sum_{j=1}^M \{1 + \ln \sigma_j^2(\mathbf{x}_n) - \mu_j^2(\mathbf{x}_n) - \sigma_j^2(\mathbf{x}_n)\}$$

This closed-form expression is possible because **both distributions are Gaussian**.

STUDENT Q&A: Is the analytical KL formula only valid because both distributions are Gaussian?

The lecturer confirmed: “Yes, we do the derivation with q being Gaussian and the prior being $\mathcal{N}(\mathbf{0}, \mathbf{I})$. If you don’t make that assumption, there are different issues and you need much more data.”

8 The Reparameterisation Trick

This is one of the key technical innovations of the VAE and was heavily discussed across both lectures.

8.1 The Problem

Why gradients w.r.t. ϕ are problematic

- Computing the gradient w.r.t. decoder weights $\mathbf{w}$: **standard** backpropagation — no issue.
- Computing the gradient w.r.t. encoder parameters ϕ : **problematic** because changes to ϕ change the distribution $q(\mathbf{z}_n|\mathbf{x}_n, \phi)$ from which $\mathbf{z}$ is sampled. The sampling operation “blocks” backpropagation.

Fixing $\mathbf{z}_n$ to a specific sample blocks the error signal from propagating back to the encoder.

LECTURE INSIGHT:

Intuition for why sampling blocks gradients The lecturer’s explanation: “Think about it this way — you have the encoder outputting μ and σ . Then you *sample* $\mathbf{z}$ from the distribution parameterised by μ and σ . Once you sample, the value of $\mathbf{z}$ is fixed — it’s a constant. You cannot backpropagate through a constant. The sampling operation breaks the computational graph.”

8.2 The Solution

Reparameterisation Trick

If $\epsilon \sim \mathcal{N}(0, 1)$, then $z = \sigma\epsilon + \mu$ has a Gaussian distribution with mean μ and variance σ^2 . Instead of drawing $\mathbf{z}_n$ directly from $q(\mathbf{z}_n|\mathbf{x}_n, \phi)$, we draw $\epsilon \sim \mathcal{N}(0, 1)$ and compute:

$$z_{nj}^{(l)}(\mathbf{x}_n, \phi) = \sigma_j(\mathbf{x}_n, \phi) \epsilon_{nj}^{(l)} + \mu_j(\mathbf{x}_n, \phi)$$

where $l = 1, \dots, L$ indexes samples and $j = 1, \dots, M$ indexes dimensions of $\mathbf{z}$.

This makes the dependence on ϕ **explicit** and allows gradient computation via standard backpropagation.

LECTURE INSIGHT:

The trick is “quite simple but brilliant” The lecturer praised this invention: “I don’t know if Kingma was lucky in finding this, but it’s a very nice, very original contribution. The idea is simple: instead of sampling $\mathbf{z}$ from a parameterised distribution (which blocks gradients), you sample ϵ from a *fixed* distribution $\mathcal{N}(0, 1)$ and then compute $\mathbf{z}$ as a deterministic function of ϵ , μ , and σ . The stochasticity is now in ϵ , not in the path from ϕ to $\mathbf{z}$.”

8.3 Full Mini-Batch Objective

Mini-batch ELBO with reparameterisation

For a mini-batch of size B :

$$\mathcal{L} = \sum_{k=1}^B \left\{ \frac{1}{2} \sum_{j=1}^M (1 + \ln \sigma_{kj}^2 - \mu_{kj}^2 - \sigma_{kj}^2) + \frac{1}{L} \sum_{l=1}^L \ln p(\mathbf{x}_k | \mathbf{z}_k^{(l)}, \mathbf{w}) \right\}$$

where $\mathbf{z}_k^{(l)}$ has components $z_{kj}^{(l)} = \sigma_{kj} \epsilon_{nj}^{(l)} + \mu_{kj}$.

Key: k indexes mini-batch elements, j indexes latent dimensions, l indexes Monte Carlo samples for estimating $p(\mathbf{x}_k | \mathbf{z}_k, \mathbf{w})$.

Setting $L = 1$ in practice

In practice, the number of Monte Carlo samples L per data point is typically set to **1**. This gives a noisy estimate, but since it is part of stochastic gradient optimisation, it leads to efficient training overall.

LECTURE INSIGHT:

Why $L = 1$ works The lecturer acknowledged this seems counter-intuitive: “It’s a bit odd because we are doing Monte Carlo with just one sample! But in practical terms it still works because you are sampling many times across different mini-batches. Stochastic gradient descent is already very noisy, and adding one more source of noise doesn’t significantly hurt — the overall optimisation still converges.”

9 Training Procedure Summary

VAE Training Algorithm

For each mini-batch during training:

1. For each data point $\mathbf{x}_n$ in the mini-batch, run the **encoder** to obtain $\mu_j(\mathbf{x}_n, \phi)$ and $\sigma_j^2(\mathbf{x}_n, \phi)$.
2. **Sample** from the approximate posterior using the reparameterisation trick: draw $\epsilon \sim \mathcal{N}(0, 1)$ and compute $z_j = \sigma_j \epsilon_j + \mu_j$.
3. Use these samples $\mathbf{z}$ as inputs to the **decoder** to evaluate the ELBO.
4. Compute gradients of the ELBO w.r.t. $\mathbf{w}$ and ϕ using **automatic differentiation** and update both via backpropagation.

LECTURE INSIGHT:

Training intuition The lecturer’s summary: “Unlike a standard autoencoder where the output is a value and we want output = input, here we have a **probability distribution** as output. We want the probability of the input $\mathbf{x}$ to be *maximum* — that is, $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$ should be as high as possible. We achieve this by maximising the ELBO, which also makes the approximate posterior q better.”

10 Generation After Training

Using a VAE after training

After training:

1. The **encoder is discarded**.
2. New data are generated by sampling $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ from the prior.
3. The sampled $\mathbf{z}$ is fed into the **decoder network** to produce $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$.

LECTURE INSIGHT:

Why the encoder can be discarded The lecturer explained: “This is precisely why we chose the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ — it does **not** depend on ϕ . So at generation time, we don’t need the encoder at all. We just sample from the standard normal and run the decoder. If we had used a conditional prior $p(\mathbf{z}|\mathbf{x})$, we would still need the encoder and some input $\mathbf{x}$ to generate.” The data $\mathbf{x}$ can be anything: images, protein sequences, or any data representation.

STUDENT Q&A: After training, do I just sample ϵ from a normal distribution and pass through the decoder?

Yes, but note that ϵ here plays the role of $\mathbf{z}$ sampled from the prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$. You sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and feed it to the decoder. The encoder is no longer needed.

STUDENT Q&A: Can VAEs be used outside of image generation?

The lecturer confirmed VAEs can be used for “anything — images, sequences of amino acids, text (though not ideal for text), risk analysis in finance.” He described it as “a good way for learning probability distributions” applicable to many domains.

11 Conditional VAE and Extensions

Conditional VAE (brief mention)

If we use $p(\mathbf{z}_n|\mathbf{x}_n, \phi)$ instead of $p(\mathbf{z})$ in the KL term, we obtain a **Conditional VAE**. In this case:

- We are forcing the output to match a different prior for every input $\mathbf{x}$.
- We need values of $\mathbf{x}$ to generate new samples.
- It is useful when we want class-conditional generation.

Planned extensions (not fully covered)

The slides mention the following topics that were planned but not fully delivered:

- VAE code tutorial (PyTorch implementation).
- Conditional VAE (Bishop & Bishop, p. 576).
- Posterior collapse (Bishop & Bishop, p. 577).

12 Potential Exam Questions

EXAM ALERT:

Classic exam question: AE vs VAE The lecturer directly posed this as a potential exam question in the 03-17 lecture: “If I ask you: what is the difference between a classic autoencoder and a variational autoencoder?” The expected answer centres on: (1) VAEs learn a *probability distribution* over the latent space, not just a deterministic mapping; (2) This makes VAEs true generative models; (3) VAEs are trained by maximising the ELBO, not just minimising reconstruction error.

EXAM ALERT:

Derive or explain the ELBO The ELBO derivation was presented in full detail across both lectures. Be prepared to:

- Reproduce the 5-step derivation from $\ln p(\mathbf{x}|\mathbf{w})$ to $\mathcal{L} + \text{KL}$.
- Explain why $\mathcal{L}$ is a *lower* bound ($\text{KL} \geq 0$).
- Explain the dual optimisation property: maximising ELBO improves both p and q .

EXAM ALERT:

Explain the reparameterisation trick This was described as “one of the most important things to remember about VAE training.” Be able to:

- Explain *why* sampling blocks gradient computation.
- State the trick: $z_j = \sigma_j \epsilon_j + \mu_j$ with $\epsilon \sim \mathcal{N}(0, 1)$.
- Explain *why* this resolves the problem (stochasticity moved to ϵ , which is independent of ϕ).

EXAM ALERT:

Write the KL divergence for Gaussians The closed-form KL divergence between the approximate posterior and the standard normal prior:

$$\text{KL}(q\|p) = -\frac{1}{2} \sum_{j=1}^M (1 + \ln \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

Know that this is only analytically tractable because both q and $p(\mathbf{z})$ are Gaussian.

EXAM ALERT:

Why use $\mathcal{N}(\mathbf{0}, \mathbf{I})$ as the prior? Expect a question asking why we choose the standard normal prior. Key points: global latent space, no need for encoder at generation time, smooth representations, avoids overfitting.

EXAM ALERT:

Why $L = 1$ works in practice Know the justification: although one Monte Carlo sample is noisy, it is embedded in stochastic gradient descent which is already noisy, and convergence is still achieved.

EXAM ALERT:

VAE training procedure Be able to list the complete training procedure: encoder $\rightarrow \mu, \sigma \rightarrow$ reparameterisation trick $\rightarrow$ decoder $\rightarrow$ evaluate ELBO $\rightarrow$ backpropagate gradients to both $\mathbf{w}$ and ϕ .

EXAM ALERT:

Generation after training Know that: (1) the encoder is discarded, (2) sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, (3) pass through decoder to get $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$. Know *why* the encoder is not needed (the prior does not depend on ϕ).

13 References

13.1 Cited on Slides

- Bishop & Bishop. *Introduction to Deep Learning*. Springer, 2024. Chapter 19 (VAEs). Figures used in the slides.
- Kingma, D.P. and Welling, M. *An Introduction to Variational Autoencoders*. Foundations and Trends in Machine Learning. Now Publishers, 2019. arXiv:1906.02691. [**Highly recommended by the lecturer — 120 pages, very close to the presentation style used in class.**]
- Kingma, D.P. and Welling, M. “Auto-Encoding Variational Bayes.” ICLR 2014. [**The seminal VAE paper; ~80,000 citations.**]
- Vincent, P. et al. “Stacked Denoising Autoencoders...” JMLR 2010.

13.2 Mentioned in Lectures Only

- PyTorch VAE tutorial: https://github.com/Jackson-Kang/Pytorch-VAE-tutorial/blob/master/01_Variational_AutoEncoder.ipynb
- The lecturer noted that the Bishop & Bishop textbook presentation is “much more difficult” than the Kingma & Welling monograph, and recommended the latter for self-study.